ENERGY AND CARBON DISCLOSURE · OFFICIAL · PUBLIC

What our software consumed, January to March 2026

Calendar quarter, 1 January to 31 March 2026, 90 days. Measured by Perf Sentinel 0.31.0.

Energy, operational	Carbon, location-based	Avoidable with known fixes
18.42 kWh	1.91 $kgCO_2eq$	0.34 $kgCO_2eq$
4 150 000 requests measured, 98.8 % of those declared	a directional estimate, true value likely between 0.95 and 3.82	18 % of measured I/O, at the fixed canonical threshold

DECLARED: SIGNED

This document carries the machine-readable report it was rendered from, as an attachment, with the signature Example SAS's pipeline placed on it in a public log. The printed code alone proves nothing: the proof is that signature, checked against the signer Example SAS publishes. Page 5 shows how, in five minutes.

document code
sha256: 6caf 6951 8a14 913b 6822 5804 08eb d877 4837 ffa2 e917 ef4a c47e 6e14 51de 6abd

signed 2026-10-16T21:15:34Z · Rekor log index 3 284 840 363

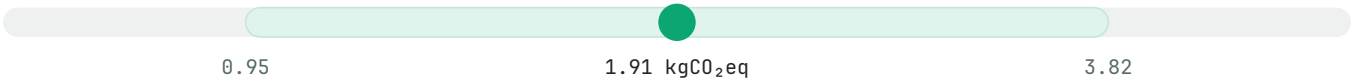
Three numbers, and how sure we are of them

Energy is measured from the work the services did. Carbon applies the carbon intensity of the electricity grid to that energy. Avoidable is what a known class of defects, repeated database and HTTP calls, wasted.

Energy, operational	Carbon, location-based	Efficiency score
18.42 kWh	1.91 $kgCO_2eq$	82.1 / 100
over 90 days, 4 150 000 requests	a directional estimate, see the band below	higher is better, computed by the engine

The carbon figure and its uncertainty

The method is a directional estimate with a ~2x bracket. The true value is likely to sit anywhere in the shaded band.



Avoidable waste, two readings of the same data

Canonical, N+1 from 2		0.34 kg · 18 %
Operational, N+1 from 5		0.19 kg · 10 %

The canonical figure uses a threshold fixed in the measuring binary, so the operator cannot shrink it by tuning. The operational figure uses the operator's own threshold and is what their dashboard shows. Both are published so a reader can compare.

How much of the quarter was actually measured



Runtime coverage 91 %

164 of 180 scoring windows carried per-service energy attribution. The other 16 fell back to a proportional share of I/O operations. An official report needs at least 75 %.

Temporal coverage 96.7 %

87 of 90 days carried measured traffic, the largest gap was 2 days. Quiet days lower this figure too, so it is a floor on activity, not an uptime claim.

Where these numbers go in European climate reporting

Not reported

Which services consumed what

Three of the four applications Example SAS declared were measured. Per-service carbon is operational only. The count of anti-patterns is published, their detail is not, by choice of granularity.

Application	Requests	Share of energy	kWh	$kgCO_2eq$	Efficiency	Anti-patterns
Checkout	2 100 000	51 %	9.40	0.98	78.0	20
Catalog	1 500 000	36.4 %	6.70	0.70	86.0	22
Billing	550 000	12.6 %	2.32	0.23	84.5	5
legacy-batch	Excluded by the operator: OTel instrumentation pending, scheduled Q2. Declared, not measured.					
Total	4 150 000		18.42	1.91	82.1	47

Scope, as declared and as measured

Declared by the operator	4 applications · 1 environment (prod) · 4 200 000 requests in the period
Derived by the engine	3 applications measured · 4 150 000 requests measured · 98.8 % coverage
Region	eu-west-3 · carbon intensity from electricity_maps

Reading the efficiency score

The score rewards services whose I/O is useful work rather than repetition. A lower score does not mean a larger footprint: a service can score higher than another while consuming less, because it carries less traffic, not because it is better written.

How the figures were produced, and what they are not

Specification	ISO/IEC 21031:2024 · conformance core-required
Measuring binary	perf-sentinel 0.31.0 · SPECpower table 2026-04-24 (CCF aligned) · energy models cloud_specpower, io_proxy_v3
Detectors enabled	n_plus_one_sql, n_plus_one_http, redundant_sql, redundant_http (core, required) · slow_sql, slow_http, excessive_fanout, chatty_service, pool_saturation, serialized_calls
Detectors disabled	none
Runtime calibration	not applied this period · 164 runtime windows, 16 fallback windows
Archive	Not reported

Disclaimers, exactly as the report carries them

1. Directional estimate, not regulatory-grade.
2. Approximate uncertainty bracket: ~2x multiplicative.
3. Not suitable for CSRD or GHG Protocol Scope 3 reporting.
4. Methodology: ISO/IEC 21031:2024 (SCI).
5. Per-service carbon includes operational emissions only; embodied carbon (SCI M term) is reported in the aggregate total but not attributed per service.
6. Energy and carbon attribution per service is runtime-calibrated when the window's energy_model is non-empty; archives without runtime energy attribution fall back to proportional I/O share.
7. Temporal coverage: 87/90 declared days carried measured traffic (96.7%, largest gap 2 days). Archiving is traffic-gated, so days without measured traffic, including legitimately quiet periods, lower this figure. It is a lower bound on activity, not a daemon-uptime guarantee.

What this document is

A public transparency report produced by an open-source tool, published by Example SAS on its own responsibility. It is not an audited inventory and does not make a CSRD submission. The scope is IT compute observed through application traces: no end-user devices, no buildings.

Five minutes to know it was not altered

The check is on the report attached inside this PDF, not on its layout.

Claim	Checked by
The figures match document code sha256: 6caf 6951 8a14 913b 6822 5804 08eb d877 4837 ffa2 e917 ef4a c47e 6e14 51de 6abd	perf-sentinel when this PDF was rendered, and you
The signed attestation names that same code	perf-sentinel when this PDF was rendered, and you
Example SAS's pipeline signed it, in the public Sigstore log	you, with the command below
The measuring binary is an official Perf Sentinel release	you, with the command below
An auditor reviewed the figures	nobody, this report is not audited

Attached to this PDF

- ☐ **report.json** · source — 7 KB
  `sha256 e3fabfd83d3c85665d32e9038420a3c71bf313db362f0a891edebc7c226cf5fd`
- ☐ **attestation.intoto.jsonl** · supplement, names the document code — 1 KB
  `sha256 0c2d7a4e8f02c8d7b28012dacf66dae2b4b1f11596cf7fcabf0036ed0ea1f798`
- ☐ **bundle.sig** · supplement, Sigstore bundle — 11 KB
  `sha256 1ce844ebcf95f9636944dd9a15d407e6a0826b5ee9058b887f81bf16b8d5e78b`

If you do not run commands

1. Ask Example SAS for the page where it publishes its reports. It should list the document code of each report.
2. Compare that code with the document code printed in full on page 1 and in the table above, every group. A different code means a different report. A matching code is only as good as the page you compared it with.
3. For proof, hand this PDF to anyone with a terminal. The steps below need free tools and no account.

If you do

```
perf-sentinel verify-hash --report report.pdf \
  --verify-binary <perf-sentinel-binary> \
  --expected-issuer https://token.actions.githubusercontent.com \
  --expected-identity \
  'https://github.com/Institut-du-Numerique-Responsable/perf-sentinel/.github/workflows/sample-disclosure.yml@refs/heads/main'
# TRUSTED    code, signature and binary all check out
# UNTRUSTED  something was altered, or the signer is not Example SAS's
```

It reads the attachments out of the PDF itself. TRUSTED needs cosign and gh on the PATH, perf-sentinel alone stops at PARTIAL.

What a tampered copy looks like

A changed figure in the attached report changes the document code, so it fails step 2. A forger who prints the new code instead fails the command: the signed attestation names the original code, and the signature sits in a public, append-only log that neither Example SAS nor the forger can edit. The code covers the attached report.json only: a PDF editor can change the printed figures without changing it, so compare the figures with the attached report.